

Chapter 1

Software Architecture Case Study — Vineyard Weather Station

ENEL712 Embedded Systems Design — Lecture 12

Summary

Designing modular embedded software using the NZ vineyard weather station as a case study: hardware block diagram, modules, source files, functions, and an event-driven state-machine architecture.

Topics Covered

- Project specification
- Hardware block diagram overview
- Software architecture and modularization
- Final modular structure
- Driver abstraction levels
- State-machine + event-driven architecture

1.1 Project Specification

- Capabilities: monitor and record temperature, humidity, rainfall, sunlight hours.
- Power: self-powered or designed to minimise battery replacement.
- Connectivity: wireless link to the owner's house.
- Initial briefs are typically short — MCU choice, wireless tech, sample rate, and physical size are unspecified.

1.2 Hardware Block Diagram Overview

- MCU: ATmega 328P.

- Sensors: TCN75A temperature (I²C), HTS221 humidity (I²C), Hydreon RG-11 rainfall (pulse counter), Excelitas VT90N1 LDR sunlight.
- Power: 7.4 V 500 mAh Li-Po → 3.3 V via Microchip MCP1824T LDO.
- Storage/comms: Microchip 25AA1024 EEPROM (SPI), Digi XBee Pro ZigBee (UART).
- Other: JTAG interface for debugging, system clock.

1.3 Software Architecture and Modularization

1.3.1 1. Defining modules

- Modules are abstract groupings (Communications, Controller, UI).
- First step from spec to design — usually 3–5 modules.

1.3.2 2. Multiple source files

- Each module = one or more .c files; files allow independent build/test.
- Encapsulation: internal data/funs not declared in the header stay local.
- Example: calcs.c, gps.c, lcd.c, setup.c.

1.3.3 3. Functions and logic

- Each file holds many small functions, each doing one job.
- Granularity: don't write one giant readGPS — split into decodeTime, decodeLatLong, etc.
- Aim for 3–6 functions per source file.

1.4 Final Modular Structure for the Weather Station

Module	Source file	High-level functions	Low-level / driver functions
Sensors	sensors.c	readAllSensors(data)	readTemp(), readHum(), readLight(), readRain(), readADC(channel), I2CRead/Write
Communications	comms.c	txMeasmnt(data)	txChecksum(), readyToSend(), UART_TxTime(), UART_TxInt(), UART_Tx()
Memory	memory.c	writeMeasmnt(data), readMeasmnt()	getCapacity(), freeMemoryLeft(), clearMemory(), writeByte(), readByte(), SPIExchange()
Main	main.c	main() state machine	ISR, setup, UART_Rx

Central data structure: timestamp, temperature, humidity, rainfall, sunshine.

1.5 Driver Abstraction Levels

Level	Example
Low-level	Direct register access, e.g., SPDR = byte
Mid-level	spi_transfer(byte) hides register details
High-level	readTemp() calls mid-level I ² C functions and applies calibration

1.6 State-Machine + Event-Driven Architecture

- Battery-powered → must spend most time asleep.
- main() implements the core loop as a state machine (e.g., SLEEP/SENSE/TX/SLEEP).
- ISRs handle WDT wakeup, UART data ready, etc., so the CPU can sleep otherwise.

1.7 Use Cases & Applications

- The vineyard station pattern (Sleep → Sense → Process → Transmit → Sleep) generalises to any battery-powered logger: livestock trackers, weather stations, environmental loggers.
- I²C for slow sensors + SPI for high-throughput storage is a typical low-power node split.
- WDT-only wakeups + an event-driven main FSM is the dominant architecture for sub-100 μ A average current designs.

1.8 Worked Example: Time Budget per Wake

Suppose the station wakes once every 5 minutes for a 2 s active window:

- Time online per day: $2 \text{ s} \times (24 \times 60 / 5) = 576 \text{ s} \approx 9.6 \text{ min}$ — under 0.7 % duty cycle.
- If active current is 200 mA and sleep current 30 μ A, $I_{\text{avg}} \approx 0.0067 \times 200 + 0.9933 \times 0.03 \approx 1.37 \text{ mA}$.
- On a 2,000 mAh Li-Po, lifetime $\approx 2,000 / 1.37 \approx 1,460 \text{ h} \approx 60 \text{ days}$.
- Levers to extend: cut active time (process less locally), drop wake rate, switch to a lower-current radio.

1.9 State-Machine Skeleton (main.c)

```
typedef enum { S_SLEEP, S_SENSE, S_LOG, S_TX } state_t;

int main(void) {
    setup();
    state_t s = S_SLEEP;
    measurement_t m;

    for (;;) {
        wdt_reset();
        switch (s) {
            case S_SLEEP: enter_power_down(); /* WDT will wake */ s = S_SENSE; break;
            case S_SENSE: m = readAllSensors();                      s = S_LOG;
        }
        break;
    }
}
```

```

        case S_LOG:    writeMeasmnt(&m);                s = S_TX;
break;
        case S_TX:    txMeasmnt(&m);                s = S_SLEEP;
break;
    }
}
}

```

1.10 Failure Modes & Mitigations

- Wireless TX never returns ACK → bound the retry loop and drop to log-only mode.
- EEPROM write failure → checksum each record; if the checksum reads back wrong, retry on a different cell.
- Sensor I²C lockup → re-init the bus periodically; if a transaction times out, reset the affected device line.
- Brown-out mid-write → store settings in two slots and pick the higher-versioned valid one on boot.

Key Definitions

MCU

Single-chip computer integrating CPU, memory, and peripherals for real-time control.

I²C

Synchronous two-wire bus (SCL/SDA) supporting multiple devices via addressing.

SPI

Synchronous, full-duplex four-wire serial protocol with master/slave roles.

UART

Asynchronous serial protocol relying on agreed baud rate and frame format.

Encapsulation

Public declarations live in headers; private details in .c — restricts access.

Modularization

Splitting software into manageable abstract components.

FSM

Finite State Machine architecture pattern.

Non-Functional Requirements

Constraints on how well the system performs (timing, reliability, power).

Duty Cycle

Fraction of time spent in the active state vs total period.

Periodic Wake

Pattern of using a low-power timer (WDT or RTC) to wake the MCU at a fixed cadence.

Power Budget

Allocation of average current per subsystem against the total battery capacity.

Sources

- Lecture 12.pdf